

Course Structure



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Course Structure					
Course Code	BSC10050				
Course Category	PF				
Course Title	Microprocessor				
Teaching Scheme	Lectures	Tutorials	Laboratory / Practical	Project	Total
Weekly load hours	3		2		5
Credits	3		1		4
Assessment Code	TL3				

Assessment Structure



Dr. Vishwanath Karad
MIT WORLD PEACE
UNIVERSITY | PUNE
 TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Type of Course	Assessment Scheme Code	Description L-T-P-J-C	CCA1	MT	CCA2	LCA1	LCA2	LCA3	TE	Total
Theory and Lab Course 3	TL3	3-0-1-0-4	10	25	10	5	5	5	40	100

Course Outcomes



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Course Outcomes:

On completion of the course, student will be able to–

1. Understand the block diagram of 8086 Microprocessor
2. Understand the Instruction set of 8086
3. Write an assembly language program
4. Understand the advanced Microprocessor

Course Contents



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Course Contents:

Unit 1: 8086 Architecture

10 Hours

Evolution of Microprocessors and types, 8086 Microprocessor, Salient features, Pin descriptions, Architecture of 8086 - Functional Block diagram, Register organization, Concepts of pipelining (5 stage), Memory segmentation, Physical memory addresses generation., Minimum mode and Maximum Mode of 8086

Unit 2: Instruction Set of 8086 Microprocessor

15 Hours

Addressing modes, Data transfer instructions, Arithmetic Instructions, Logical Instructions, Bit manipulation instructions, String Operation Instructions, Program control transfer or branching Instructions, Process control Instructions, examples based on Instruction Set

Course Contents



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Unit 3: 8086 Assembly Language Programming

10 Hours

Introduction to Assembly Programming and Assembler Directive

Assembly program on Addition, Subtraction, Multiplication and Division

Sum of Series

Smallest and Largest numbers from array

Sorting numbers in Ascending and Descending order

Block transfer

String Operations - Length, Reverse, Compare, Concatenation, Copy

Unit 4: Advanced Processor

10 Hours

Introduction to Pentium Processor, Feature of Pentium Pro Processor Architecture of Pentium Pro Processor,

Software Model of Pentium Pro , Pipelining of Pentium Processor , Classification of Advanced Processors

Course Contents - Lab



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Lab Experiments for Microprocessor

Assembly language program for arithmetic operations

Assembly language program to convert the given decimal number to binary, octal number system

Assembly language program to find the largest and smallest among the given numbers

Assembly language program to arrange the given numbers in ascending and descending order

Assembly language program to find the square of a number

Assembly language program to find sum of an array

Resources



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Learning Resources:

Reference Books:

- 1) Microprocessor & interfacing (programming & hardware) Revised Second Edition By Douglas V. Hall
Tata McGraw Hill
- 2) Microprocessor Architecture By B.Ram
- 3) Microprocessor Architecture, Interfacing & Programming By Mohammad Raffiqzamman

Supplementary Reading:

- 1) The Intel Microprocessors : Barry B. Brey- Pearson Education Asia
- 2) The Pentium Microprocessor : James Antonakos

Content

- ✓ Addressing modes
- ✓ Data transfer instructions with examples
- ✓ Arithmetic Instructions with examples
- ✓ Logical Instructions with examples
- ✓ Bit manipulation instructions with examples
- ✓ String Operation Instructions with examples
- ✓ Program control transfer or branching Instructions with examples
- ✓ Process control Instructions with examples



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

ADDRESSING MODES OF 8086



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

- ✓ The addressing modes are the ways of specifying an operand in an instruction.
- ✓ In 8086, the addressing modes are broadly categorized into two groups, i.e. data addressing modes and address addressing modes.
- ✓ Data addressing modes are for defining a data operand in the instruction.
- ✓ Address addressing modes are the ways of specifying a branch address in control transfer instructions.

Data Addressing Modes of 8086



- ✓ The microprocessor 8086 has all the 5 data addressing modes available in 8085, i.e. implied, register, immediate, direct and register indirect. The register indirect addressing mode in 8086 works with SI, DI, BX and BP registers.
- ✓ Apart from these data addressing modes, 8086 has five more addressing modes. These are:
 1. Base addressing mode
 2. Index addressing mode
 3. Based indexed addressing mode
 4. Based indexed with displacement addressing mode
 5. String addressing mode

Immediate addressing mode : In immediate addressing mode, the operands are specified within the instruction itself. The immediate operand can only be the source operand. For example,

MOV AX, 2500H

Here, the immediate data is 2500H.

Implied/Implicit addressing mode : In this addressing mode, the operands are specified explicitly in the instruction. For example, STC ; Set carry flag
CLD ; Clear the direction flag

Data Addressing Modes of 8086



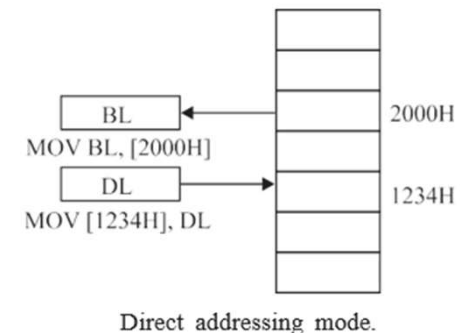
Register addressing mode : Most 8086 instructions can operate on the 8086's general purpose register set. The content of a register can be accessed by specifying the name of the register as an operand to the instruction.

MOV AX, BX
MOV DL, AL
MOV SI, DX

The 8- and 16-bit registers are the valid operands for this instruction. The only restriction is that both operands must be of the same size. Instructions using the registers are shorter and faster than those that access memory.

Direct addressing mode : It may be defined as the addressing mode in which the address of the memory is specified in the instruction itself. In this addressing mode, the instruction consists of a 16-bit memory address or an 8-bit IO address. The 16-bit memory address is always written inside the square brackets. For example, the instruction **MOV BL, [2000H]**, transfers the content of the memory location 2000H in the BL register.

By default, all the direct addressing mode point in the data segment. The segment override prefix is to be used before address if we have to point any other memory segment. For example, to access location 4321H in the extra segment ES the instruction will be of the form **MOV AX, ES: [4321H]**. Similarly, to access this location in the code segment, the instruction will be **MOV AX, CS: [4321H]**.



Data Addressing Modes of 8086



Register indirect addressing modes : This addressing mode is also used in concern with memory and IO. In this addressing mode, the memory address is specified by some pointer, index or base registers. These registers are written inside the square brackets. For example :

MOV DX, [BX]

MOV DX, [BP]

MOV DX, [SI]

MOV DX, [DI]

The [BX], [SI], and [DI] modes use the DS segment by default. The [BP] addressing mode uses the stack segment (SS) by default.

Base addressing mode : In this mode, 8-bit or 16-bit displacement is added to the contents of a base register (BX or BP). The resulting value is a pointer to location where data resides. In this addressing mode, the memory location is calculated by adding the signed 8-bit or 16-bit displacement to either BX or BP register.

For example if BX = 2000H, the instruction is **MOV AL, [BX + 15]**

In this example, the contents of the memory location 200FH (2000H + 0FH (equivalent to decimal 15)) is transferred to AL register. The maximum 8-bit displacement can be ± 127 and the maximum 16-bit displacement can be ± 32767 .

Data Addressing Modes of 8086



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Index addressing mode : This addressing mode is similar to base addressing mode with the difference that, the 8-bit or 16-bit displacement is added to the contents of an index register (SI or DI). In this addressing mode, the memory location is calculated by adding the signed 8-bit or 16-bit displacement to either SI or DI register.

SI $\pm$ 8-bit displacement
Memory location = SI $\pm$ 16-bit displacement
DI $\pm$ 8-bit displacement
DI $\pm$ 16-bit displacement

Data segment is the default segment for this addressing mode. As with the register indirect and base addressing modes, the segment override prefixes can be used to specify a different segment:

MOV AL, CS: DISP [SI]
MOV AL, SS: DISP [DI]

Data Addressing Modes of 8086

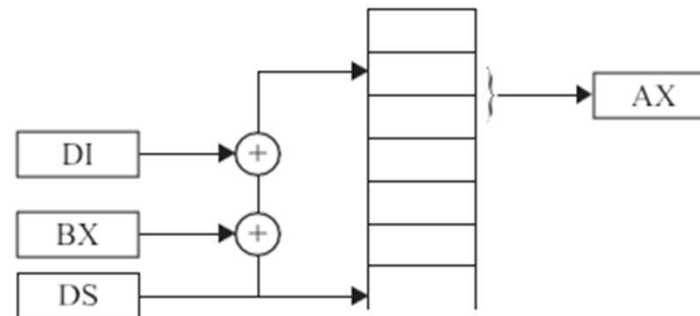


Based indexed addressing mode : The based indexed addressing modes are simply the combinations of the register indirect addressing modes. In based indexed addressing mode, the contents of a base register (BX or BP) are added to the contents of an index register (SI or DI), the resulting value is a pointer to location where data resides.

For example, if $BX = 2000H$ and $SI = 5400H$, the instruction is

`MOV AL, [BX + SI]`

In this example the contents of the memory location $7400H$ ($2000H + 5400H$) is transferred to AL register.



Calculation of offset address in base index addressing mode.

Data Addressing Modes of 8086



Based indexed with displacement addressing mode : In this addressing mode, the offset address is generated by the sum of Base register and Index registers along with 8-bit or 16-bit displacement. In this addressing mode, 8-bit or 16-bit displacement is added to the contents of a base register (BX or BP) and index register (SI or DI); the resulting value is a pointer to location where data resides. For example, if BX = 2000H and SI = 5400H, the instruction is

MOV AL, [BX + SI + 15]

In this example, the contents of the memory location 740FH (2000H + 5400H + 0FH (equivalent to decimal 15)) is transferred to AL register.

String addressing modes : This mode uses index registers. The string instructions automatically assume SI to point to the first byte or word of the source operand and DI to point to the first byte or word of the destination operand.

The segment register for the source is DS and may be overridden. The segment register for the destination must be ES and cannot be overridden.

The contents of SI and DI are automatically incremented by clearing DF (Direction Flag) to 0 by CLD instruction or automatically decremented by setting DF to 1 by STD instruction.

Instruction Set of 8086 - Data Transfer



Data Transfer Instructions : The data transfer instructions copy values from one location to another. These instructions include MOV, XCHG, XLAT, LDS, LEA, LES, PUSH, PUSHF, POP, POPF, LAHF, SAHF, IN, and OUT. **No Flag will change in data transfer instruction.**

- ✓ **MOV destination, source** (Move source to destination) : The MOV Instruction copies the second operand (source) to the first operand (destination) without modifying the contents of the source. In true sense, these are not the data transfer instructions but data copy instruction because the source is not modified. MOV AL,[5055H], MOV AX,[5055H]
- ✓ **IN port address (Input data from input port)**
Example : AL, 8-bit port address AL, DX
AX, 8-bit port address AX, DX

This instruction transfers a byte or word from a port to the accumulator register. The port address is specified by the source operand, which can be DX or an 8-bit constant. If the port address is of 8-bit then direct addressing will be used.

Example : IN AX, 04H; Move the content of the port number 04H and 05H to AL and AH

IN AL, 70H; Move the content of the port number 70H to AL.

If the port number is of 16-bit, then indirect addressing will be used and DX will be the default register to specify the port number. To note that the DX register is not enclosed in square brackets like memory addressing.

Example : IN AX, DX; Move the content of the port number specified in DX and DX + 1 to AL and AH
IN AL, DX; Move the content of the port number specified in DX to AL.

Instruction Set of 8086 - Data Transfer



✓ *OUT destination, source (Output data to output port)*

This instruction transfers a byte or word to an output port from the accumulator register. The port address is specified by the destination operand. Port number may be of 8-bit or 16-bit. For 16-bit port address DX register will be used and for 8-bit port address an 8-bit constant will be used.

Example : OUT 04H, AX; Transfer the content of the AX register to the port 04H and 05H. OUT 70H, AL; Transfer the content of the AL register to the port 70H.

OUT DX, AX; Transfer the content of the AX register to the port specified by DX and DX + 1.

OUT DX, AL; Transfer the content of the AL register to the port specified by DX.

✓ *LDS Rd, M: Load pointer using DS*

This instruction is used to load the double word stored in memory into specified register and DS register. The general purpose register is loaded from the lower-order word of the memory operand and the segment register DS from the higher-order word. The memory may be specified by any of the memory-related addressing mode. This instruction is very useful in string instructions where the source is always DS:SI. For example, LDS BX, [4405H] ; load the content of two memory location from DS+4405H to BL and BH register.

Instruction Set of 8086 - Data Transfer



- ✓ **LEA(Load effective address) Rd, M** : This instruction is used to load the effective address (offset) of the source memory operand into the specified destination register. For example,

```
MOV SI, 1000h ; Load base address into SI
LEA BX, [SI+20h] ; Load effective address (1000h + 20h) into BX
MOV AX, [BX] ; Move the value at the calculated address into AX
```

- ✓ **XCHG destination, source** ; Exchange values of two operands. The order of the XCHG's operands does not matter but both operands should be of the same size. For example, if BL = 15 and AH = 20, then after the instruction XCHG BL, AH the content of BL will be 20 and AH will be 15. It does not support segment registers. The direct exchange of data between memory locations is illegal. Both operands should be a general-purpose register.

- ✓ **LAHF no operands (Load flag)**

Load the AH register by the lower byte of the flag register which contains the status flag, i.e. carry, parity, auxiliary carry, zero, and sign flags. By this instruction, we can see the contents of the status flag.

- ✓ **SAHF no operands (Store flag)**

Store the AH register in the lower byte of the flag register which contains the status flag, i.e. carry, parity, auxiliary carry, zero, and sign flags. By this instruction we can modify the contents of the status flag.

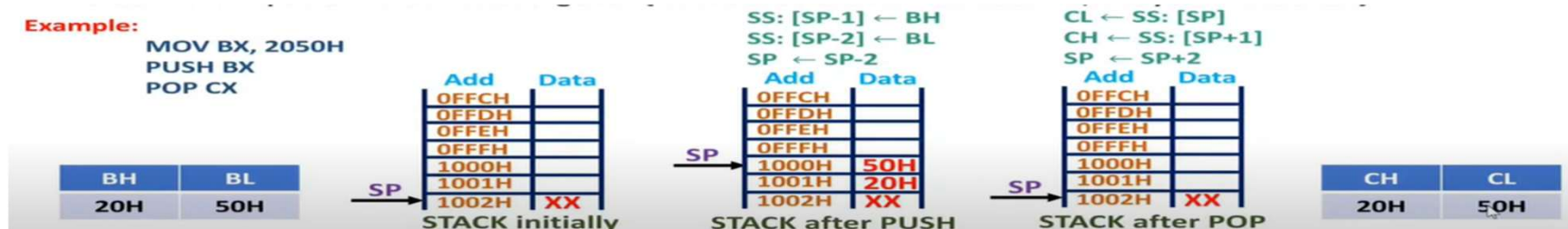
Instruction Set of 8086 - Data Transfer



PUSH Push content on to stack top : Push instruction pushes the source operand onto the stack. After the PUSH instruction, the content of SP is decreased by 2 and the source value is copied to SS:SP. **The PUSH instruction always operates on words.** The operands in the PUSH instruction can be a memory location, a general purpose 16-bit register, or a segment register. PUSH SP copies the value of SP after the push.

For example, if the content of the AX register is 1234H, then after the PUSH AX instruction the 12h is loaded on [SP] and 34H is loaded on [SP - 1] and SP is further decremented by 1 and becomes $SP = SP - 2$. The flag register remain unchanged after this instruction. **This doesn't follow immediate addressing mode.**

POP Pop-off contents from top of stack : This instruction pops the top of the stack into the destination operand. This means that the value at SS:SP is copied to the destination operand and SP is increased by 2. The destination operand can be a memory location, a general purpose 16-bit register, or any segment register except CS. For example, if the top of the stack contains 12H and 34H, then after the instruction POP BX, the SP is increased by 1 and the content of that location is loaded into BL and SP again is incremented by 1 and the content of that location is loaded into BH. **The flag register remain unchanged after this instruction.** PUSHF and POPF is also used to copy the content of flag register into/ from the stack.



Instruction Set of 8086 - Data Transfer



- ✓ **XLAT** : This is the translate instruction which is used to translate a value from one coding system to another with the help of a lookup table. In this instruction BX is by default one of the registers which points to the starting location of the lookup table. This BX register must be initialized before the XLAT instruction. AL is the second register used in this instruction and contains the unsigned value which is to be translated from the table. The pointed table value will be transferred in the AL register after the execution of the instruction. This instruction do not required any operand. DS is the by default segment which can be override by segment override prefix. The XLAT instruction will perform the operation $AL = DS:[BX + \text{unsigned } AL]$. For example, consider the program

```
LEA BX, 2000H
```

```
MOV AL, 20
```

```
XLAT
```

In this program, AL will be loaded with the content of the memory location pointed by $[2000 + 20]$.

Instruction Set of 8086 - Data Transfer



Example

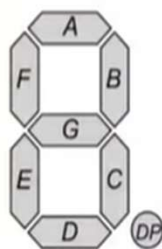
```
MOV BX, 2050H
MOV AL, 05H
XLAT
```

Data Segment Look Up Table

	Add	Data
	204FH	
BX →	2050H	0'
	2051H	1'
	2052H	2'
	2053H	3'
	2054H	4'
BX+AL →	2055H	5'

- Here 0', 1', 2', 3', 4', 5', ... are equivalent codes of 0, 1, 2, 3, 4, 5, ... , respectively.

Equivalent codes of seven Segment Display



DP	G	F	E	D	C	B	A	DATA	INPUT
0	0	1	1	1	1	1	1	3FH	0'
0	0	0	0	0	1	1	0	06H	1'
0	1	0	1	1	0	1	1	5BH	2'
0	1	0	0	1	1	1	1	4FH	3'
0	1	1	0	0	1	1	0	66H	4'
0	1	1	0	1	1	0	1	6DH	5'
0	1	1	1	1	1	0	1	7DH	6'
0	0	0	0	0	1	1	1	07H	7'
0	1	1	1	1	1	1	1	7FH	8'
0	1	1	0	1	1	1	1	6FH	9'



Instruction Set of 8086 - Arithmetic



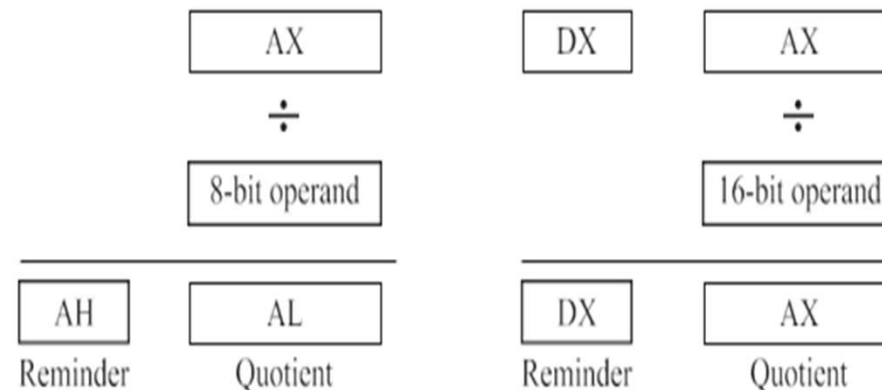
Arithmetic Instructions : The 8086 provides many arithmetic operations: addition, subtraction, negation, multiplication, division/modulo (remainder), and comparing two values. The instructions that handle these operations are ADD, ADC, SUB, SBB, MUL, IMUL, DIV, IDIV, NEG, INC, DEC, CBW and CWD. **In the arithmetic instructions, the flag register is modified.**

- ✓ **ADD/ADC destination, source** : These instructions add a data from source operand to a data from destination and save the result in the destination operand. Segment registers cannot be used as an operand in ADD instruction. Memory to memory and IO to memory addition is also not permitted. ADC instruction adds the source operand to the destination operand along with the carry flag. Example : ADD AX, 4321H , ADC AX, BX
- ✓ **SUB/SBB destination, source** : SUB instruction subtracts the source operand from the destination operand and stores the result in the destination operand and SBB instruction subtracts the source from the destination along with the value of the carry flag. Example : SUB AX, 4321H , SBB AX, BX
- ✓ **INC operand** : This instruction increment the destination operand by 1. The INC instruction is more compact and often faster than the comparable ADD instruction because it is a one- byte instruction.
- ✓ **DEC operand** : This instruction decrement the destination operand by 1. This instruction differs with the SUB by 1 instruction in the way that the DEC instruction does not affect the carry flag whereas the SUB instruction modifies the carry flag. The DEC instruction is more compact and often faster than the comparable SUB instruction because it is a one-byte instruction.

Instruction Set of 8086 - Arithmetic



- ✓ **DIV Source** : This instruction divides the contents of the AX with specified source operand. The AX is the implied destination operands for 16-bit and 32-bit division. This is an unsigned operation and hence both operands are treated as unsigned operands. If the divisor is 16-bits wide, then the dividend is the DX-AX register pair. After the division the quotient will be stored into AX and the remainder into DX. When the divisor is of 8 bits, the dividend is AX. And in this case the quotient will be stored in AL and the remainder in AH. Example: DIV CL, DIV BX
- ✓ **IDIV Source** : This instruction is exactly the same as that of DIV except that here both operands are signed numbers. In both DIV and IDIV instructions, the source operand cannot be an immediate data or a segment register. All the flag bits are undefined, i.e. the value of all the flag bits may be either 0 or 1.

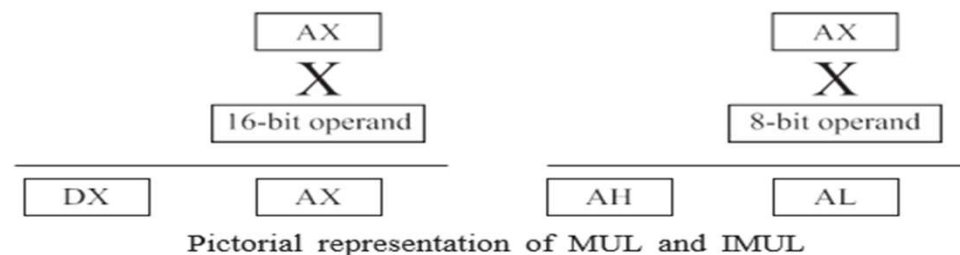


Pictorial representation of DIV and IDIV.

Instruction Set of 8086 - Arithmetic



- ✓ **MUL Source** : This instruction multiplies the contents of the AL or the AX by a specified source operand. The AL and the AX are the implied destination operands for 8-bit and 16-bit multiplication. This is an unsigned operation and hence both operands are treated as unsigned operands. In this instruction both the implied operand and the source operand must be of the same size. For a 16-bit multiplication, the implied operand will be the AX register. After the multiplication the product, which is of 32-bits, will be stored into the DX-AX register pair. For an 8-bit multiplication, the implied operand will be the AL register. After the multiplication the product, which is of 16-bits, will be stored into the AX register. If after the multiplication DX is not 0 for 16-bit operands or AH is not zero for 8-bit operands, then the carry and overflow flags will set. The A, P, S, and Z flags are undefined, i.e. the value of these flag bits may be either 0 or 1.
- ✓ **IMUL Source** : This instruction is exactly the same as that of MUL except that here both operands are signed numbers. In both MUL and IMUL instructions, the source operand cannot be an immediate data or a segment register. If the product is sign extended into DX for 16-bit operands, into AH for 8-bit operands then the carry and overflow flags are set and the remaining flag bits, i.e. the A, P, S, and Z flags are undefined, i.e. the value of these flag bits may be either 0 or 1.



Instruction Set of 8086 - Arithmetic



- ✓ **NEG Source** : This instruction produces the 2's complement of the specified operand and stored the result in the same operand. Microprocessor performs the negate (NEG) operation by subtracting the operand from 0. This is done to represent a negative number. All the flag bits are modified as per the result. Example MOV AL,15H NEG AL
- ✓ **CBW (Convert byte to word)** : CBW converts the signed value in the AL register into an equivalent 16-bit signed value in the AX register by extending the sign bit to the left.

If AL = 46H = 0100 0110 b, then after Execution of CBW instruction, AX = 0000 0000 0100 0110 b = 0046H

If AL = 96H = 1001 0110 b, then after Execution of CBW instruction, AX = 1111 1111 1001 0110 b = FF96H

Positive numbers are extended by inserting 0's ahead number and negative numbers are extended by inserting 1's ahead numbers.



- ✓ **CWD (Convert word to doubleword)** : Convert word into double word. CWD converts the 16 bit signed value in the AX register into an equivalent 32-bit signed value in DX-AX register pair by duplicating the sign bit to the left.

CWD

If AX = 9746H = 1001 0111 0100 0110 b, then after Execution of CWG instruction,

DX - AX = 1111 1111 1111 1111 - 1001 0111 0100 0110 b = FFFF - 0046H

Positive numbers are extended by inserting 0's ahead number and negative numbers are extended by inserting 1's ahead numbers.

Instruction Set of 8086 - Arithmetic



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

- ✓ **AAA (ASCII adjustment after addition)** : AAA converts the result of the addition of two valid unpacked BCD digits to a valid packed BCD number. AL register is the implied operand. The decimal numbers in ASCII have a code from 30H ("0") to 39H ("9"). When we add two ASCII numbers, the result will not be in ASCII. The AAA instruction will adjust the result of addition to a BCD digit. The instruction assumes that the add operands are proper ASCII values.

ADD AL, CL
AAA

If AL = 36H {ASCII of 6} and If CL = 37H {ASCII of 7}
After ADD AL,CL result of AL = 6DH.
After AAA, result of Carry = 1, AL = 03H

- ✓ **AAS (ASCII adjustment after Subtraction)** : AAS converts the result of the subtraction of two valid unpacked BCD digits to a single valid BCD number. AL register is an implicit operand. The two operands of the subtraction must have its lower 4-bit contain number in the range of 0 to 9. The AAS instruction then adjust AL so that it contains a correct BCD digit.

SUB AL, CL
AAS

If AL = 38H {ASCII of 8} and If CL = 32H {ASCII of 2}
After SUB AL,CL result of AL = 06H.
After AAS, result of Carry = 0, AL = 06H

Instruction Set of 8086 - Arithmetic



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

- ✓ **AAM (ASCII adjustment after multiplication)** : AAM converts the result of the multiplication of two valid unpacked BCD digits into a valid unpacked BCD number. AX is the implicit operand in AAM.

MUL AL, CL

AAM

If AL = 06H {Unpacked BCD of 6} & CL = 07H {Unpacked BCD of 7}.

After MUL AL,CL result of AX = 002AH.

After AAM, result of AX = 0402H

- ✓ **AAD (ASCII adjustment before division)** : AAD converts unpacked BCD digits in the AH and AL register into a single binary number in the AX register in preparation for a division operation.

AAD

If AX = 0607H means AH = 06H, AL = 07H

After AAD instruction, AL = 67 = 43H

After that we can perform Division instruction.

Instruction Set of 8086 - Arithmetic



- ✓ **DAA (Decimal adjust after addition)** : The DAA instruction adjusts the result of an addition to a packed BCD number. DAA converts this binary sum to packed BCD format. If the sum is greater than 99h after adjustment, then the carry and auxiliary carry flags are set. Otherwise, the carry and auxiliary carry flags are cleared. DAA works with AL register only.
- ✓ For example, two BCD values are added as if they were binary numbers and the result will be in binary and then to convert this binary sum the DAA instruction is executed to correct the result in BCD. The DAA instruction functions like AAA except it handles packed BCD values rather than unpacked. All flag bits are modified as per the result.

Add 06H with AL, If Lower Nibble > 9 or Auxiliary Carry = 1

Add 60H with AL, If Higher Nibble > 9 or Carry = 1

ADD AL,BL
DAA

AL = 20H
BL = 40H
AL = 60H

AL = 56H
BL = 23H
AL = 79H

AL = 36H
BL = 29H
AL = 5FH
06H
AL = 65H

AC = 1
AL = 38H
BL = 29H
AL = 61H
06H
AL = 67H

AL = 60H
BL = 70H
AL = D0H
60H
AL = 30H
Carry = 1

CY = 1
AL = 90H
BL = 80H
AL = 10H
60H
AL = 70H
Carry = 1

1 1
AL = 99H
BL = 99H
AL = 32H
06H
60H
AL = 98H
Carry = 1

- ✓ **DAS (Decimal adjust after subtraction)** : The DAS instruction adjusts the result of a subtraction to a packed BCD number (less than 100 decimal). DAS converts the binary result of subtraction into packed BCD. If the sum is greater than 99H after adjustment, then the carry and auxiliary carry flags are set. Otherwise, carry and auxiliary carry flags are cleared. All flag bits are modified as per the result.

Sub 06H with AL, If Lower Nibble > 9 or Auxiliary Carry = 1

Sub 60H with AL, If Higher Nibble > 9 or Carry = 1

Instruction Set of 8086 - Logical



The 8086 provides six logical instructions. The logical instructions are AND, OR, XOR, TEST, NOT and CMP. These instructions can manipulate bits, convert values, do logical operations, pack and unpack data operations.

- ✓ **NOT (Logically NOT) Operand** : This instruction complements the individual bits of the operand and save the result in the same operand. We can say that it generates the 1's complement or the NOT operation of the operand. After this instruction the flag register remain unmodified. For Example: MOV AL,39H NOT AL
- ✓ **AND (Logically AND) destination, source** : This instruction performs a bitwise Logical AND of destination operand and the source operand. The result of the operation is stored in the destination operand.

AND AL,BL

If AL = 47H = 0100 0111 b

& BL = 36H = 0011 0110 b

After execution of instruction AL = 0000 0110 b = 06H

- ✓ **TEST destination, source** : This instruction performs logical AND between all bits of the source and destination operands. In this instruction perform the logical AND operation but none of the operands is modified, it is only the ZF, SF, PF flags are modified. This instruction is used to tests specified bits of an operand and sets the flags for a subsequent conditional jump or set instruction. One of the operands contains the value to be tested. The other contains a bit mask indicating the bits to be tested.

Instruction Set of 8086 - Logical



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

- ✓ **XOR (Logically EX-OR) destination, source** : This instruction performs a bitwise exclusive OR operation between the source and destination operands. After the operation, the result is stored in the destination.

XOR AL,BL

If AL = 47H = 0100 0111 b

& BL = 36H = 0011 0110 b

After execution of instruction AL = 0111 0001 b = 71H

- ✓ **CMP (Compare) destination, source** : This instruction compares the source operand with the destination operand. Microprocessor executes this CMP instruction by subtracting the source operand from the destination operand but none of the operands are modified. The result is reflected by the flag bits. Generally the result (i.e. flag conditions) of this instruction is used for conditional control transfer instructions.

- ✓ **OR (Logically OR) destination, source** : This instruction performs a bitwise logical OR operation between the source and destination operands. The result is stored in the destination operand. After the operation, the Z, S, and P flag bits are modified.

OR AL,BL

If AL = 47H = 0100 0111 b

& BL = 36H = 0011 0110 b

After execution of instruction AL = 0111 0111 b = 77H

PF, SF & ZF affected, CF = OF = 0 & AF is undefined.

Instruction Set of 8086 - Shift



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

The 8086 provides four rotate instructions, and three shift instructions. This is the second set of instructions which are used for bit manipulation. These instructions can manipulate bits, convert values, do logical operations, pack and unpack data, and do arithmetic operations.

- ✓ **SAL/SHL (Arithmetic/logical shift left) destination, count** : The SHL (Shift Logically Left) and SAL (Shift Arithmetically Left) are used to shift the content of the first operand towards left. The encoding of these two mnemonics is the same. The second operand may be an immediate data or the counter register CL and represent the number of times the shifting takes place. While each bit of the operand shifts to left, the zeros fill vacated positions at the lower order bit. During SAL/SHL, Sign bit is not conserve i.e. sign bit may change from 0 to 1 or vice versa.

SHL DH,1

- If DH = 47H = 0100 0111 b and Carry = 1



Carry

- After Execution of instruction



Carry

- After Shift Left DH = 8EH, Carry = 0

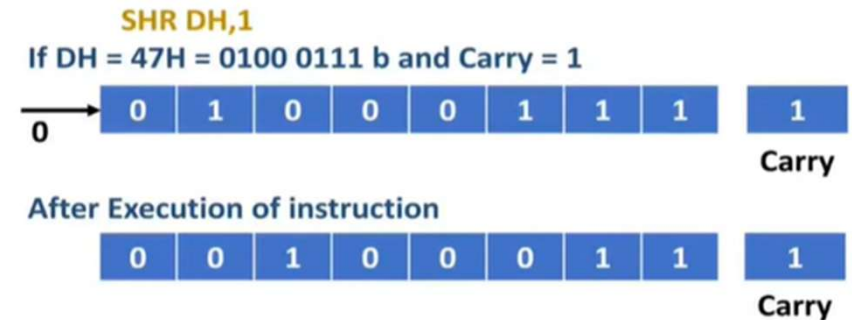
MOV CL,03H

SHL DH,CL

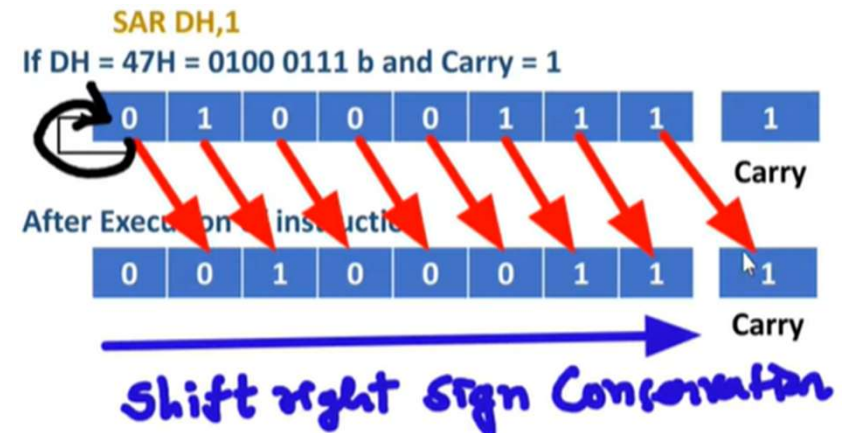
It will shift data of DH left by CL = 3 bits.

Instruction Set of 8086 - Shift

- ✓ **SHR (Logical shift right) destination, count** : Shift, logically, the content of the first operand right by the number of times specified by the second operand. The second operand may be an immediate data or the counter register CL. In SHR while each bit is shifted right and goes to the carry flag, a zero bit is inserted from the MSB side.
- ✓ **SAR (Arithmetically shift right) destination, count** : Shift arithmetically, the content of the first operand right by the number of times specified by the second operand. The second operand may be an immediate data or the counter register CL. In SAR while each bit is shifted right and goes to the carry flag, the sign bit is inserted from the MSB side. As we know that shifting a binary number towards right is equivalent to dividing that number by two, hence the SAR instruction is used to perform a signed division. Each shift to the right divides the value of the destination by two. During SAR, Sign bit is conserve i.e. sign bit may not change from 0 to 1 or vice versa.



After Shift Left DH = 23H, Carry = 1



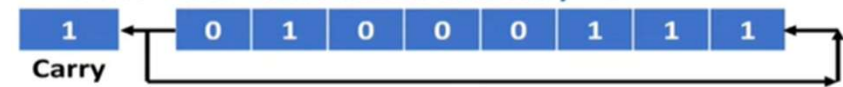
Instruction Set of 8086 - Rotate

- ✓ **ROL (Rotate left) destination, count :** The ROL instruction is used to rotate the operand 1 left. In this instruction the second operand may be an immediate value or the counter register CL. This second operand decides the number of times the rotation will take place. The ROL instruction moves the D15 or D7 bit to the D0 bit position of the operand as well as to the carry flag and all other bits shifts towards left, i.e. to the higher bit position in the operand register. The ROL instruction is similar to the RCL instruction with a difference that the carry flag bit is not part of the rotation.
- ✓ **ROR (Rotate right) destination, count :** This instruction is the same as that of RCR with a difference that the LSB will go to the MSB position as well as to the carry. Only carry and overflow flags will be modified rest will be undefined, i.e. may be 0 or 1.



ROL DH,1

- If DH = 47H = 0100 0111 b and Carry = 1



- After Execution of instruction ROL



- After Rotate Left DH = 8EH, Carry = 0
- Example

MOV CL,03H
ROL DH,CL

- It will Rotate data of DH left by CL = 3 bits.

ROR DH,1

- If DH = 47H = 0100 0111 b and Carry = 1



- After Execution of instruction ROR



- After Rotate Right DH = A3H, Carry = 1

Instruction Set of 8086 - Rotate

- ✓ **RCL (Rotate left through carry) destination, count :** The RCL instruction is used to rotate the operand1 left through the Carry Flag. In this instruction the second operand may be an immediate value or the counter register CL. This second operand decides the number of times the rotation will take place. The RCL instruction moves the D15 or D7 bit to the Carry Flag and the Carry Bit goes to the D0 bit position and all other bits shift towards left, i.e. to the higher bit position in the operand register.
- ✓ **RCR (Rotate right through carry) destination, count :** This instruction performs the rotation of the contents of operand right through the Carry Flag. In this instruction the second operand may be an immediate value or the counter register CL. This second operand decides the number of times the rotation will take place. This instruction RCR is the complement to the RCL instruction.



RCL DH,1

- If DH = 47H = 0100 0111 b and Carry = 1



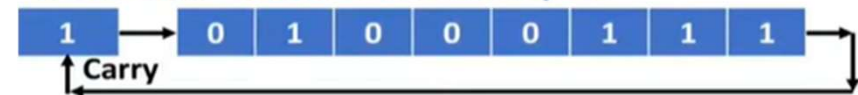
- After Execution of instruction RCL



- After Rotate Left DH = 8FH, Carry = 0

RCR DH,1

- If DH = 47H = 0100 0111 b and Carry = 1



- After Execution of instruction RCR



- After Rotate Right DH = A3H, Carry = 1

Instruction Set of 8086 - String



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

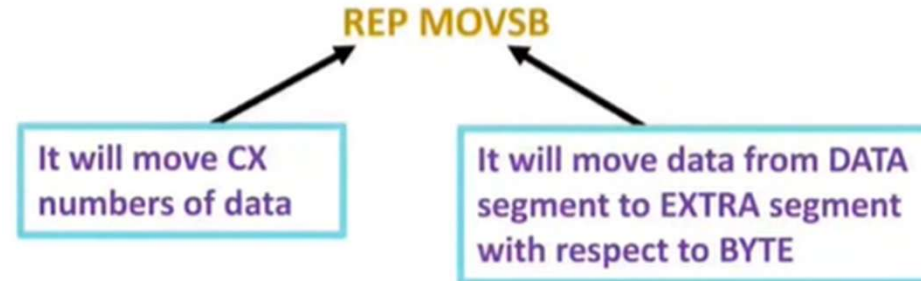
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

String instructions are the instructions used for manipulation of sequence of bytes or words that contain the alphanumeric code for characters. These instructions are used for processing the text. There are five types of string instructions in 8086. The string instructions of 8086 are MOVS (move string), CMPS (compare string), SCANS (scan string), LODS (load string) and STOS (store string). The count of string must be given in CX register. As CX is 16 bit register so you can transfer 64 KB data using string instructions.

- ✓ **MOVSB (Move string byte) / MOVSW (Move string word) no operands** : These instructions are used to transfer the string byte (MOVSB) or a word byte (MOVSW) source pointed by DS:SI to the destination location pointed by ES:DI. After the execution of the instruction, the SI and DI is incremented by one (in case of string byte) or two (in case of string word) if the direction flag is zero, otherwise the SI and DI will be decremented by the same value for byte and word string.

Example :

```
LEA SI, [2000H]
LEA DI, [3000H]
MOV CX, 0015H
REP MOVSB
```



Instruction Set of 8086 - String

- ✓ **LODSB (Load string byte) / LODSW (Load string word)** : Load AL or AX by the string byte or string word pointed by SI in the Data Segment. After the execution of the instruction, the SI is incremented by one (in case of string byte) or two (in case of string word) **if the direction flag is zero**, otherwise the SI will be decremented by the same value for byte and word string. The flag register will remain unmodified. A segment override can be given to change the source segment DS.
- ✓ **STOSB (Store string byte) / STOSW (Store string word)** : These two instructions are used to store the string data available in the AL or AX in the memory specified location by ES:DI. After the operation the DI register is either incremented or decremented by one or two depending on the status of the direction flag and according to the size of the operands. DI is increased if the direction flag is zero or decreased if the direction flag is one. STOSB and STOSW are generally used with the REP prefix.



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

LODSB



It will load AL
from string
pointed by SI

LODSW



It will load AX
from string
pointed by SI

STOSB



It will store AL
on string
pointed by DI

STOSW



It will store
AX on string
pointed by DI

Instruction Set of 8086 - String



Dr. Vishwanath Karad

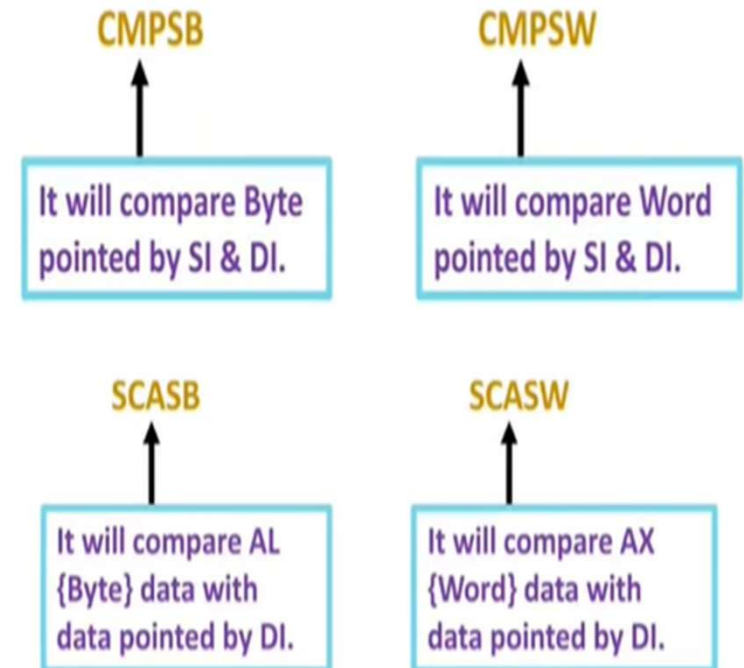
**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

✓ **CMPSB (Compare bytes string) / CMPSW (Compare words string) :**

These instructions are used to compare the two string bytes or string words stored in memory. The source string is always pointed by SI in the data segment and the destination string is pointed by the DI in the extra segment. To compare these two strings, the microprocessor performs the subtraction operation. It subtracts the destination string from the source string. After the operation, none of the operands is modified, only the flags are modified to reflect the result.

✓ **SCASB (Scan string byte) / SCASW (Scan string word) :** These instructions are used to scan a string stored in memory to find a value specified in the accumulator register. These instructions actually compare the string present in AL or AX with the string stored in memory pointed by ES:DI. The SCAS instruction is performed by subtracting each element of the destination from the accumulator contents. After the operation the flags are updated to show the comparison result.



Instruction Set of 8086 - Process Control (Prefixes)



- ✓ **REP (Repeat conditionally)** : The repeat conditionally instruction which is used as prefix to a string instruction is used to repeat a string instruction as long as condition is true and the CX is not zero. There are two conditional repeat instructions and four mnemonics. These are **REPE/REPZ** and **REPNE/ REPNZ**. Before executing any of these instructions, CX must be initialized with the maximum allowable number of repetitions. For each string element, the string instruction is performed, CX is decremented, and the zero flag is tested. Except zero, all the other flag bits are undefined.
- (i) **REPE/REPZ (Repeat till equal/Repeat till zero)** : REPE (Repeat till equal) and REPZ (repeat till zero) instructions are used to repeat a string instruction till the zero flag is set. As in these two instructions the zero flag is tested so these instructions are used with the SCAS and CMPS instructions as these are the only string instructions that modify the zero flag. For example, Repeat CMPSB, CMPSW, SCASB, SCASW instructions while $ZF = 1$ (result is Equal) or till CX reaches to zero.
- (ii) **REPNE/REPNZ (Repeat till not equal/Repeat if no zero)** : REPNE (Repeat till not equal) or REPNZ (repeat if no zero) instructions are used to repeat a string instruction till zero flag is not zero or till CX becomes zero. For example, Repeat CMPSB, CMPSW, SCASB, SCASW instructions while $ZF = 0$ (result is Not Equal) or till CX reaches zero.

Instruction Set of 8086 – Flag



These instructions are used to set or reset the individual control flag bits. No operands are associated with these instructions.

- ✓ **CLC (Clear carry)** : CLC clear the carry flag (CF) to 0. This instruction has no effect on the processor, registers, or other flags. It is often used to clear the CF before returning from a procedure to indicate a successful termination. It is also use to clear the CF during rotate operation involving the CF such as ADC, RCL, and RCR.
- ✓ **CLD (Clear direction flag)** : This instruction reset the designation flag to zero. This instruction has no effect on the registers or other flags. When the direction flag is cleared/reset SI and DI will automatically be incremented when one of the string instructions such as MOVS, CMPS, SCAS, MOVSB and STOSB executes.
- ✓ **STD (Set direction flag)** : This instruction sets the Direction flag. SI and DI will be decremented by chain instructions: CMPSB, CMPSW, LODSB, LODSW, MOVSB, MOVSW, STOSB, and STOSW.
- ✓ **STI (Set interrupt enable flag)** : This instruction sets the Interrupt enable flag. When the interrupt flag is set, maskable interrupts are enabled. If interrupts were disabled by a previous CLI instruction, pending interrupts will not be executed immediately; they will be executed after the instruction following STI
- ✓ **CLI (Clear interrupt flag)** : This instruction resets the interrupt flag to zero. No other flags are affected. If the interrupt flag is reset, the 8086 will not respond to an interrupt signal on its INTR input. This CLI instruction has no effect on the non-maskable interrupt input, NMI.
- ✓ **CMC (Complement carry)** : If the carry flag CF is zero before this instruction, it will be set to one after the instruction. If the carry flag is one before this instruction, it will be reset to zero after the instruction executes. CMC has no effect on other flags.
- ✓ **STC (Set carry)** : This instruction sets the Carry flag.

Instruction Set of 8086 – Control



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

The control transfer instructions are used to transfer the control from one memory location to another location. In 8086 there are four types of control transfer instruction, i.e. jump, call, return and interrupt. All these type of control transfer instructions may be conditional or unconditional. In case of conditional control transfer instructions, the control will be transferred only if the condition is true.

✓ **Call (subroutine/procedure) Operands (procedure name/ label/ 4-byte address) :** This instruction is used to transfer the control of execution to a subroutine or procedure. There are two basic types of CALL instructions, the Near CALL and the Far CALL.

A CALL instruction is called **Near CALL** if the procedure is in the same code segment as the main program.

When microprocessor executes the CALL instruction, the control is transferred to the procedure but before it goes to the subroutine it saves the returning address on the top of stack. The stack pointer is decremented by two and copies the offset of the next instruction after the CALL on the stack. The control is again transfer to the main program when microprocessor comes across RET instruction at the end of the procedure. This is done by popping up the offset saved on the stack back to IP.

When the procedure is stored in a code segment different from where the main program is stored, the CALL instruction is called **Far CALL**. In this case also the returning address which consists of CS:IP is stored at the top of stack. First the stack pointer is decremented by two and stored the content of CS register to the stack top. Again stack pointer is decremented by two and stored the IP contents on stack top.

Instruction Set of 8086 – Control



- ✓ **RET (Return) no operands** : This is the last instruction of a procedure. When microprocessor executes this instruction, transfer the control from the subroutine to the main program, in doing so it popped the returning address from the top of the stack. A near return is specified by the RETN and far return is specified by RETF. A near return works by popping a word into IP. A far return works by popping a word into IP and then popping a word into CS.
- ✓ **IRET (Interrupt return) no operands** : The IRET (Return from Interrupt Service Routine) transferred the control from an interrupt procedure to the main program. The IRET instruction pops IP, CS, and the flags (in that order) and resumes execution.
- ✓ **INTn (Interrupt)** : The INTn generates the software interrupts. Here in this instruction the subscript n may be from 0 to 255 and is called type of interrupt. When microprocessor executes the software interrupt INTn, the control is transferred to a predefined memory location called the vectored location. From this vectored location, the address of the ISR is fetched by the microprocessor and then the control is transferred to that ISR. When an interrupt is called, the flags CS and IP are pushed onto the stack (in that order) and the trap and interrupt flags are cleared. To return from an interrupt, use the IRET instruction.
- ✓ **INTO (Interrupt on overflow) no operands** : Interrupt of overflow flag is 1. The control is transferred to the ISR if the overflow flag is set after an arithmetic instruction. If overflow is not set, then the ISR will not be called.

Instruction Set of 8086 – Control



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

- ✓ **JMP (jump unconditionally) Operands (label / 4-byte address)** : This is an unconditional jump instruction. By this instruction, the control is transferred from one memory location to another location within the program. The 4-byte address specified as 234h:5678H. Here the first value denotes the segment and the second value shows the offset.
- ✓ The jump instruction may be conditional also. There are 30 conditional jump instructions. In these instructions the control will be transferred to the label or 4-byte address only when the condition is true. The condition may be logical condition or may be arithmetical condition. The arithmetical condition may further be either signed or unsigned. Table 4.10 shows the interpretation of these conditional jumps.

Interpretation of conditional jumps

Mnemonic	Condition tested
JC	CF = 1
JZ	ZF = 1
JNC	CF = 0
JNZ	ZF = 0
JNO	OF = 0
JNP	PF = 0
JNS	SF = 0
JO	OF = 1
JP	PF = 1
JS	SF = 1

Instruction Set of 8086 – Control



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

The different conditional jump instructions are:

- (i) **JC label** : jump if carry flag is set to 1. if $CF = 1$, then jump
- (ii) **JNC label**: jump if carry flag is set to 0. if $CF = 0$, then jump
- (iii) **JNP label**: jump if parity is odd . if $PF = 0$, then jump
- (iv) **JP label**: jump if parity is even. if $PF = 1$, then jump
- (v) **JNS label**: jump if sign flag is not set i.e. if $SF = 0$, then jump
- (vi) **JS label**: jump if sign flag is set i.e. if $SF = 1$, then jump
- (vii) **JNZ label**: jump if zero flag is not set i.e. if $ZF = 0$, then jump.
- (viii) **JZ label**: jump if zero flag is set i.e. if $ZF = 1$, then jump
- (ix) **JO label**: jump if overflow flag is set i.e. if $OF = 1$, then jump
- (x) **JNO label**: jump if overflow flag is not set i.e. if $OF = 0$, then jump

Instruction Set of 8086 - Process Control (Prefixes)



This group of instructions (except for NOP) generally deals with interactions in multiprocessor situations.

- ✓ **ESC (Escape)** : The ESC instruction is primarily used for external hardware synchronization, especially with coprocessors. By using special signal lines, a coprocessor can distinguish between data fetches and instruction fetches. This allows the coprocessor to decode the instruction stream in parallel with the 8086.
- ✓ **LOCK** : This instruction prevents the other processor from acquiring the buses during the execution of an instruction. This instruction is used as a prefix. For example, suppose the microprocessor is executing the instruction MOV AX, 1000H, and in the mean time the DMA controller sends a request for the buses, then the microprocessor is forced to suspend the execution of the MOV instruction and releases the buses for the DMA controller. But if the MOV instruction is prefixed with the LOCK instruction like LOCK MOV AX, 1000H, then the processor will not release the buses till the MOV instruction is complete. The processor will release the buses after the execution of this instruction and before the execution of the next instruction. The LOCK prefix will activate the LOCK pin of the microprocessor.
- ✓ **REP (Repeat unconditionally)** : This is a prefix instruction to a string instruction. This instruction repeats the string instruction by the number of times indicated by CX. The execution of the string instruction stops when CX is decremented to 0. The REP instruction is generally used with MOVS and STOS string instructions. Except zero all the other flag bits are undefined.

MOV CX, FFFFH
CLD
REP MOVS

Instruction Set of 8086 - Process Control (Prefixes)



- ✓ **WAIT (Wait)** : The WAIT instruction suspends the microprocessor execution and the processor enters into the wait state. The processor will remain in wait state until a signal (TEST) is received which indicates that the coprocessor has finished a simultaneous operation. This instruction is used when the microprocessor is used in maximum mode configuration and specifically when a math coprocessor is present in the multiprocessing environment system. This instruction prevents the math coprocessor instruction from modifying a memory location that is being modified at the same time by a processor instruction. FWAIT is an equivalent instruction in the math coprocessor's instruction set.
- ✓ **HLT (Halt)** : This instruction stops the execution of the microprocessor and force the processor to enter into the wait state. The processor comes out of this wait state only when an interrupt restarts execution at the instruction following HLT or by the external RESET signal.
- ✓ **NOP (No operation)** : This instruction performs no operation. NOP can be used for timing delays or alignment.

MCQ

Processor gets the address of the next instruction to be processed from

- (a) Instruction register (b) Instruction counter
- (c) Program counter (d) Program register

For string instructions in 8086, the by default pointer for DS is

- (a) DI (b) SI
- (c) BP (d) SP

The SP register is typically used for accessing

- (a) Strings (b) Memory
- (c) Stack (d) Data segment

What are the names of the 4-segment registers?

- (a) Data, Index, Code, Stack (b) Stack, Index, Extra, Code
- (c) Stack, Data, Base, Counter (d) Stack, Extra, Code, Data

In 8086 microprocessor , the address bus is _____ bit wide

- (a) 12 bit (b) 10 bit
- (c) 16 bit (d) 20 bit



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

MCQ



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Given that AL contains the ASCII code of an uppercase letter, it can be converted to lowercase by

- (a) ADD AL, 32 (b) SUB AL, 32
- (c) OR AL, 1101 1111 (d) AND AL, 0010 0000

The instruction MOV AX, [BX] is an example of

- (a) Indirect addressing (b) Indexed addressing
- (c) Direct addressing (d) Based addressing

The CMP instruction modifies the

- (a) Program counter (b) Instruction register
- (c) Flags register (d) Segment register

Given that the BL register contains 1111 0000, the effect of the following instruction OR BL, 0000 1111 is to

- (a) Clear BL (b) Store 1111 1111 in BL
- (c) Store 0000 1111 in BL (d) Leave BL unchanged

A data movement instruction will

- (a) Modify the status register (b) Modify the stack pointer
- (c) Modify the program counter (d) Transfer data from one location to another

MCQ



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

The 16 bit flag of 8086 microprocessor is responsible to indicate _____

- (a) the condition of result of ALU operation (b) the condition of memory
(c) the result of addition (d) the result of subtraction

The register AX is formed by grouping _____

- (a) AH & AL (b) BH & BL
(c) CH & CL (d) DH & DL

The BIU contains FIFO register of size _____ bytes

- (a) 8 (b) 6
(c) 4 (d) 12

The 1 MB byte of memory can be divided into _____ segment

- (a) 1 Kbyte (b) 64 Kbyte
(b) 33 Kbyte (d) 34 Kbyte

INC destination increments the content of destination by _____

- (a) 1 (b) 2
(c) 3 (d) 4